Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «АПО ЭВМиС»
Тема: «**Изучение аппаратно-программной архитектуры процессоров семейства Intel – макроопределения**»

Выполнил:
Студент 4-го курса
Группы АС-66
Гончерёнок К.А.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучение приемов разработки макроопределений использования их в программах.

Задание: переместить заданный символ, если он находится в строке, в начало строки.

Текст программы:

```
; NASM x86-64, Linux syscalls

%define MAX_LEN 80

%macro PRINT 2
    mov eax, 1                  ; sys_write
    mov edi, 1                  ; stdout
    lea rsi, [%1]
    mov edx, %2
    syscall
%endmacro

; Перемещает первое вхождение символа в начало строки.
; Параметры: адрес строки и операнд байта символа.
%macro MoveCharToFront 2
    push rax
    push rbx
    push rcx
    push rdx
    push rsi
    push rdi

    mov byte [found_flag], 0
    lea rsi, [%1]
    mov rdi, rsi
    mov dl, %2

%%scan:
    mov al, [rsi]
    test al, al
    jz %%done
    cmp al, dl
    je %%found
    inc rsi
    jmp %%scan

%%found:
    mov byte [found_flag], 1
    cmp rsi, rdi
    je %%done
    mov rcx, rsi
    sub rcx, rdi
    mov rbx, rsi
%%shift:
    mov al, [rbx-1]
    mov [rbx], al
    dec rbx
    dec rcx
    jnz %%shift
    mov [rdi], dl

%%done:
    pop rdi
    pop rsi
    pop rdx
    pop rcx
    pop rbx
    pop rax
```

```
%endmacro

section .data
    prompt_in   db 'Введите строку (до 80 символов, Enter - конец):', 10
    prompt_in_len equ $ - prompt_in
    prompt_chr  db 'Введите искомый символ:', 10
    prompt_chr_len equ $ - prompt_chr
    msg_before  db 'Исходная строка: '
    msg_before_len equ $ - msg_before
    msg_after   db 'Результат:       '
    msg_after_len equ $ - msg_after
    msg_nofind  db 'Символ в строке не найден, строка не изменена.', 10
    msg_nofind_len equ $ - msg_nofind
    newline     db 10

section .bss
    input_buf   resb 256
    str_buf     resb MAX_LEN + 1
    str_len     resq 1
    target      resb 1
    found_flag  resb 1

section .text
global _start

_start:
    PRINT prompt_in, prompt_in_len
    PRINT prompt_chr, prompt_chr_len

    ; Один read позволяет корректно разобрать обе строки из stdin.
    xor eax, eax                ; sys_read
    xor edi, edi                ; stdin
    lea rsi, [input_buf]
    mov edx, 256
    syscall
    test rax, rax
    jle .empty_input
    mov r12, rax                ; число прочитанных байтов

    ; Копируем первую строку, максимум 80 символов.
    xor ecx, ecx                ; индекс во входном буфере
    xor r8d, r8d                ; длина строки
.copy_string:
    cmp rcx, r12
    jae .no_target
    mov al, [input_buf + rcx]
    cmp al, 10
    je .after_first_line
    cmp al, 13
    je .after_first_line
    cmp r8, MAX_LEN
    jae .skip_long_string
    mov [str_buf + r8], al
    inc r8
    inc rcx
    jmp .copy_string

.skip_long_string:
    inc rcx
.skip_to_newline:
    cmp rcx, r12
    jae .no_target
    mov al, [input_buf + rcx]
    cmp al, 10
    je .after_first_line
```

```
    cmp al, 13
    je .after_first_line
    inc rcx
    jmp .skip_to_newline

.after_first_line:
    inc rcx
    ; Пропускаем CR/LF, включая пару CRLF.
.skip_line_breaks:
    cmp rcx, r12
    jae .no_target
    mov al, [input_buf + rcx]
    cmp al, 10
    je .consume_break
    cmp al, 13
    jne .get_target
.consume_break:
    inc rcx
    jmp .skip_line_breaks

.get_target:
    mov [target], al
    jmp .terminate_string

.no_target:
    mov byte [target], 0
    jmp .terminate_string

.empty_input:
    xor r8d, r8d
    mov byte [target], 0

.terminate_string:
    mov byte [str_buf + r8], 0
    mov [str_len], r8

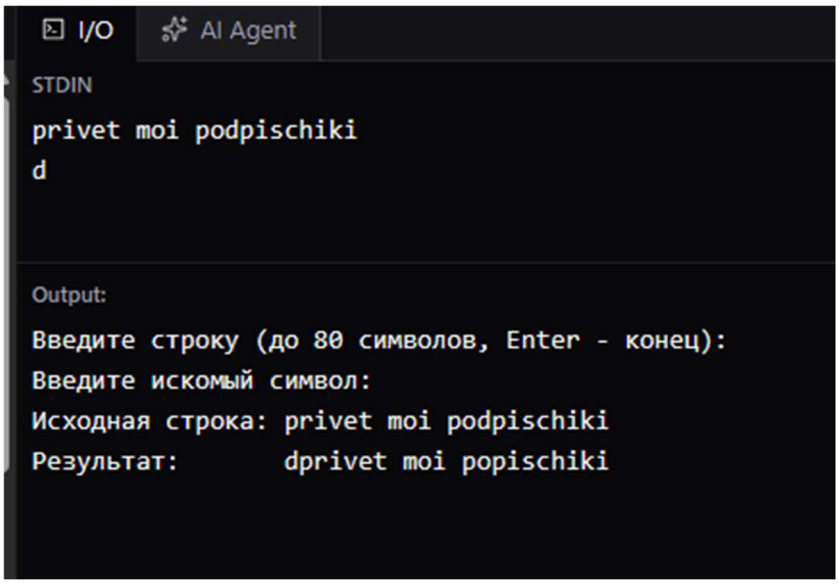
    PRINT msg_before, msg_before_len
    mov eax, 1
    mov edi, 1
    lea rsi, [str_buf]
    mov rdx, [str_len]
    syscall
    PRINT newline, 1

    MoveCharToFront str_buf, [target]

    PRINT msg_after, msg_after_len
    mov eax, 1
    mov edi, 1
    lea rsi, [str_buf]
    mov rdx, [str_len]
    syscall
    PRINT newline, 1

    cmp byte [found_flag], 0
    jne .exit
    PRINT msg_nofind, msg_nofind_len

.exit:
    mov eax, 60                 ; sys_exit
    xor edi, edi
    syscall
```



Вывод: Я изучил приемы разработки макроопределений и использования их в программах.